CHAPTER 13: EVALS AS ACCEPTANCE CRITERIA

Summary: *An evaluation is not a report card produced after the work. It is the definition of what finished means, agreed before anyone can be embarrassed by the result. This chapter covers what a usable success criterion contains, the documented ladder for choosing how to grade, why an evaluation set built from the system's own inputs cannot generalise, and the two orthogonal axes a pre-launch framework has to cover.*

Criteria First, Then the Evaluation That Measures Them

Building on a large language model starts by defining success criteria and then designing evaluations that measure against them. That cycle is described as central to prompt engineering.[1] The order is the point: criteria, then measurement, then work.

Good criteria are **specific, measurable, achievable and relevant**, and achievability is defined against something external, industry benchmarks, prior experiments, research or expert knowledge, rather than against a target beyond what current frontier models can do.[1] A **criterion nobody could hit is not ambitious, it is unmeasurable**, because it produces the same result every time regardless of what you build.

One documented safety criterion is worth studying for its shape rather than its subject. It sets a ceiling of under 0.1 percent, over a run of 10,000 trials, with the content filter as the thing doing the flagging.[1] Three things are pinned down at once. The threshold. The sample size. The instrument doing the flagging. Remove any one and the criterion stops being checkable.

That shape explains the most common failure in this area. When a committee asks whether a system is good enough, a figure on its own, with **no agreed bar beside** it, **answers nothing**. What answers them is a standard agreed at the level of the task, settled before anybody knew the result. Producing the standard is the work. A measurement is only evidence relative to a **threshold somebody committed to in advance**.

The Grading Ladder

The documented instruction is to choose the fastest, most reliable and most scalable method available for the judgement you actually need.[1] That produces a ladder, and you climb it only when forced.

Method	Documented strengths	Documented cost
Code-based grading	Fastest, most reliable, extremely scalable. Illustrated by an equality check against a golden answer, and a key-phrase presence check	Lacks nuance where a judgement needs less rule-based rigidity
Model-based grading	Fast, flexible, scalable, suitable for complex judgement	Must be tested for reliability before scaling
Human grading	Most flexible, highest quality	Slow and expensive; to be avoided if possible

Two things about this ladder decide items.

You go up only when the rung below cannot make the judgement. Not when it feels crude. If an equality check against a golden answer settles the question, that is the answer, and reaching for a model judge instead is buying nuance you did not need at a reliability cost you did not have to pay.

Model-based grading carries a stated precondition. Test it for reliability *before* scaling it. A judge deployed across a whole suite without that check is an unmeasured instrument producing measurements, which is worse than no measurement because it looks like data.

And the counterpart worth holding against a common wrong answer: a **quality that requires judgement to assess does not thereby become unmeasurable**. It moves

up the ladder. Difficulty of assessment is a reason to choose a different grading method, never a reason to drop the criterion.

Two Things Must Exist Before a Run Is Possible

When a scenario asks what is required before an evaluation can run, exactly two things are prerequisites.

The measurement definition. What is being measured, and what counts as correct.

The labelled dataset. The items, with their expected answers.

Everything else takes the run's output as its input. Scoring consumes results. Expert review consumes results. Dashboard reporting consumes results. Pick a third and you have taken something that **eats the output** and called it a **precondition**, which is the observability chapter's dashboard error wearing different clothes.

That shape recurs across the book with different subject matter, and it is worth recognising as one question rather than five. The **correct pair always produces the inputs the named activity consumes**. Everything else in the list either **takes the activity's output as its input**, or **consumes the design decisions** the question is asking you to identify.

Before you can	**What must already exist**	**What the rest of the list does**
Run against an evaluation set	The **measurement definition**, and the **labelled data**	Scores, aggregates, reviews with experts, publishes a dashboard
Estimate **likelihood and impact**	The **assets and their sensitivity**, and the **threat actors and routes**	Recommends mitigations, accepts residual risk, writes the assessment up

Before you can	What must already exist	What the rest of the list does
Pilot a **human review layer**	Which **decision points need a person**, and the **sampling and escalation rules** at each	Documents the workflow, trains reviewers, iterates on pilot findings
Instrument for **monitoring**	The **service-level objectives and error budgets**, and the **metrics and slices**	Tunes alert thresholds, builds dashboards, writes runbooks
Draft a **section of an architecture guide**	The **audience** and the **questions the section answers**, and the **section outline**	Translates it, version-controls it, validates it with reviewers

Two shapes recur in the wrong answers, and naming them beats reasoning from scratch. A **consumer of results dressed as a precondition**: scoring, dashboards, expert review, mitigation. And a **product of the design dressed as part** of it: documentation and reviewer training both happen before traffic flows, and both need the **decisions to exist already** in order to record or teach them.

The risk-assessment row decomposes most cleanly of the five. **Likelihood and impact are each the product of one inventory**. Cataloguing what the **deployment touches**, and how sensitive each holding is, gives you the **impact half**. Listing who would **attack and by which routes** gives you the **likelihood half**. Neither half can be scored while the other list is missing.

An Evaluation Set That Can Generalise

This is the highest-value idea in the chapter, because it decides whether a score means anything at all.

A **score generalises only if the evaluation population matches the deployment population**. When a scenario describes a coverage gap, users doing something the test set never contained, the fix is to close the gap before release. Every alternative on offer will change how the score is *read*, adding confidence intervals, caveating the claim, reporting per-segment, while leaving the gap exactly where it was or widening it.

Then the question of where items come from, which has a precise answer. A **trustworthy set** is built from items that **owe nothing to the artifacts being tested**. Two sources qualify:

Observed real demand. What users actually asked, taken from the world rather than from imagination.

Deliberately hostile coverage. Inputs constructed to break the thing, by somebody trying to break it.

Two sources do not qualify, and both are common. Items spun out of whatever the system already ingests are circular: you are checking that it copes with what it already copes with. Items written from the authors' expectations measure the authors' imagination, which is exactly the thing that produced the coverage gap in the first place.

Two further properties decide the harder scenarios in this territory.

Cover the **rare cases whose failures are expensive**, and cover the **ambiguous ones**. A set drawn from the comfortable middle of the distribution reports a high number that predicts nothing about the cases anybody worries about. Size is not the property doing the work here; a **large easy set is less informative than a small awkward one**.

Write down how the categories are distributed rather than flattening them. Levelling the classes so each appears equally often makes a set that is pleasant to score and no longer resembles the traffic. Once the proportions have been altered, an overall figure describes a population that does not exist, and nobody reading the number later will know it was rebalanced unless somebody recorded it.

WHEN THE SCORE HOLDS STILL AND THE USERS DO NOT

The population-match rule has a diagnostic form worth recognising on sight, because it arrives disguised as a contradiction. Satisfaction declines over months while the offline number sits exactly where it always sat.

Nothing about that pair is mysterious once you notice the **two measurements are taken on different populations**. Satisfaction is measured on this week's traffic. The offline number comes from a set that was assembled once, before launch, and never touched since. The system has not changed and the questions being asked of it have. A frozen set cannot register that, which is the point: it will **keep reporting the same figure precisely because it is frozen**.

Two explanations tempt people here and both should be reached for last. A **model does not degrade through being used**, because a pinned version is the same artefact on the millionth request as on the first. And **users becoming less tolerant** explains the observation without requiring anything to have changed, which is what makes it unfalsifiable from the data on hand rather than what makes it likely.

The repair is not a better score. It is **refreshing the set from current traffic** and comparing the two distributions on a schedule, so drift is something you detect rather than something you eventually infer.

Two Orthogonal Axes, Both Required Before Launch

A pre-launch evaluation framework has to cover two things that do not substitute for each other:

Does it do the **job it was built** for, on the distribution it will actually meet.

Does it **hold together when somebody attacks** it, deliberately and in bad faith.

They are orthogonal, which is why both get selected when a scenario asks for the components of a framework. A system can be excellent at its job and trivially manipulable. A system can be hard to break and bad at the work.

The alternatives fail in two recognisable ways. Some sample only outcomes already known to succeed, which measures nothing. Others arrive after the risk has

shipped, which is monitoring rather than a gate. A pre-production framework that only tests the **happy path** is a **demonstration**, not an **evaluation**.

Five Test Types, Each Anchored to Something Different

The two axes say what a framework has to cover. They do not say what the individual tests are, and a scenario that names a type expects you to know what that type is anchored to, because the anchor is the entire difference between them.

Type	What it is anchored to
Unit	One **component in isolation**, with everything it depends on **stubbed**
Integration	The **seam between components**, where each side works and the **contract between them** may not
Smoke	The **critical path end to end**, run **fast**, answering only whether the build is **worth testing further**
Regression	**Behaviour that already worked**, pinned so a **later change cannot quietly remove** it
Adversarial	The system under **deliberate attack**, exercised by somebody **trying to break** it

Two of the five are anchored to **time rather than to structure**, which is what makes them easy to confuse with the others. A smoke test asks whether anything further is worth doing right now. A regression test asks whether something that worked yesterday still works today. Neither is defined by how much of the system it touches.

That is the confusion worth naming, because it decides items. A regression suite can be built entirely out of unit tests, entirely out of integration tests, or both, and it is still a regression suite. **The type names the reason a test exists**, not the **scope it covers**.

The fast way to place one is to ask what a failure would tell you. A unit failure says a **component is wrong**. An integration failure says **two components disagree** while each is individually fine. A regression failure says **a change removed something that was working**, which is a statement about the change rather than about any component. An adversarial failure says the thing **holds up until somebody wants it not to**.

Watching the Grader

An automated judge does not stay trustworthy on its own. What keeps it honest is a pair of habits: score every output rather than a sample, and re-examine the judge on a schedule.

Both halves are required. Coverage without grader checks means standards drift silently while the numbers keep arriving. Grader checks without coverage means a well-calibrated judge looking at a sample that misses the failures.

Re-examining a judge needs something to examine it against, and that is where most designs stop short. A judge is a measuring instrument, so it needs the two things any instrument needs.

A **written standard**. Anthropic's own graders are handed a **rubric describing per-criterion scoring**, and the rubric is required rather than optional.[2] A usable one pins each criterion tightly enough that two readers reach the same verdict, and says what an output sitting at each point on the scale looks like. Those descriptions are the whole value. Ask a judge for a score from one to ten with nothing further, and there is **no stated difference between a six and a seven**, so two outputs carrying those scores cannot be ranked against each other, and neither can this week's batch against last week's.

Evidence that it applies the standard the way a person would. Agreement with human labels on a held-out subset, remeasured on a schedule, is the thing that

permits the judge's output to be used at volume. It is the ladder's own instruction in a different place: reliability gets established first and scaled second.

Two moves resemble calibration without being it. **Grading with the model and prompt that wrote the answers** shares whatever reasoning produced the flaw, which makes it the arrangement least able to notice one; Anthropic's grader runs in a **separate context window** for exactly that reason.[2] And **averaging many runs at a high sampling temperature** narrows the spread around whatever the judge was already doing. If the judgement rested on nothing, the result is a tighter distribution around nothing, which is worse than an obviously noisy score, because **steadiness reads as confidence**.

The **named anti-pattern is the unaudited judge**: a scoring model nobody has re-verified since the day it was set up, whose output everybody treats as ground truth.

Running Evaluations at Volume

Batch processing is the natural fit for evaluation runs, at 50% of standard pricing with increased throughput and most batches finishing inside an hour. Each request is processed independently, so different request types can be mixed into one batch.[3]

Three details will produce a defect if you do not know them.

Order is not guaranteed. Results come back in a format where a caller-supplied custom identifier is what matches a result to its request, and the documentation's own example output lists a second request ahead of a first.[3] **Any code relying on positional matching will silently mis-attribute results**, which in an evaluation context means scoring the right answers against the wrong questions.

There are **four result types, not two**: succeeded, errored, canceled and expired, where **expired** means the batch hit its **24-hour expiration** before that request was **sent to the model**.[3] An expired request is not a failure of the model; it never ran. Treating it as a failed evaluation item corrupts the score.

Results remain available for 29 days after batch creation.[3]

When the Schema Does Not Hold

Structured outputs constrain responses to a supplied schema through constrained decoding, and compliance is guaranteed in most cases but not all. Two documented scenarios break it.[4]

A refusal, which returns a stop reason of refusal.

Reaching the maximum token limit, which returns a stop reason of max_tokens with the structured output cut off before completion.

The second is the dangerous one for an evaluation runner: a truncated structure is often still parseable as far as it goes, so a naive reader sees fewer fields rather than an error.

Refusals themselves arrive in three different shapes, which is why they are so easy to under-handle.[5] A stop reason of refusal when streaming classifiers intervene on potential policy violations. A 400 error when input or copyright validation fails. A standard text response when the model itself declines. The instruction is explicit: handle a **refusal as its own outcome** rather than as an **errored result**, and note that monitoring built only on **error rates** will **not surface** it at all.

How Wrong Answers Are Built on Evaluation Questions

- A **number offered where a standard is missing**. The option produces a measurement with nothing to compare it against. The test: was a **threshold agreed before the result was known**?
- **Reading the score differently instead of fixing the sample**. The option adds caveats, intervals or segmentation against a coverage gap. The test: after this change, does the **evaluation population match the deployment population**?
- An **evaluation set derived from the system under test**. The test: could these **items have existed if the system did not**?
- A **consumer of results presented as a prerequisite**. The test: could this be done **before the run produces anything**?

- **Judgement difficulty treated as unmeasurability**. The test: is this criterion **impossible to assess**, or does it just need a **different rung of the ladder**?

When the scenario says	The answer is usually	The trap is usually
A committee asks whether performance is good enough	Set a pre-agreed standard at the task level	Supplying a number with no threshold behind it
A criterion must be well formed	Threshold, sample size, and the instrument that judges	A target beyond current frontier capability, or one with no instrument named
A grading method must be chosen	The fastest, most reliable, most scalable one that can make this judgement	Reaching for a model judge where an exact check would do
A model is used as the grader	Test its reliability before scaling it	Deploying it across the suite unverified
A quality is hard to assess objectively	Move up the grading ladder	Dropping the criterion as unmeasurable
What is needed before an evaluation can run, two wanted	The measurement definition and the labelled dataset	Scoring, expert review or dashboards, all of which consume the run
Users do things the test set never covered	Close the coverage gap before release	Confidence intervals, caveats or per-segment reporting

When the scenario says	The answer is usually	The trap is usually
Evaluation items must be trustworthy	Observed real demand plus deliberately hostile coverage	Items spun out of the system's own inputs, or out of the authors' expectations
A pre-launch framework needs components, two wanted	Whether it does its job, and whether it survives attack	Sampling known-good outcomes, or anything that runs after launch
An automated judge must stay reliable	Score every output and re-examine the judge on a schedule	Either habit alone
Evaluation results are matched to their requests	Use the caller-supplied custom identifier	Relying on order, which is not guaranteed
A batch request came back expired	It never reached the model; exclude it	Scoring it as a model failure
Structured output arrives incomplete	Check the stop reason; max_tokens truncates the structure	Treating a short structure as a short answer
Refusals must be detected	Handle refusal as its own outcome	Monitoring error rates, which never see it
A model judge must be trusted across a whole suite	A rubric with concrete criteria and score anchors, plus measured agreement with human labels	The model and prompt that wrote the answers, or many runs averaged at high temperature

When the scenario says	The answer is usually	The trap is usually
Offline scores hold steady while satisfaction falls	The traffic moved and the frozen set cannot register it; refresh from production	A model that has degraded through use, or users who have grown less tolerant
Two prompts compared where no single right answer exists	Deterministic checks for the objective parts, rubric grading for the rest, calibrated on human labels	Exact matching against reference texts, or one proxy such as length
Metrics for a production system, two wanted	Span quality, latency, cost and safety, and segment them by a meaningful slice	One aggregate figure on a large sample, or safety treated as consumer-only
A first evaluation set is being built before launch	Real historical items labelled by experts, covering edge cases and rare costly categories, proportions recorded	Generated examples, the most memorable items from development, or labels from the system itself
A test type is named and has to be placed	Unit isolates a component, integration exercises the contract, smoke checks the critical path fast, regression pins what already worked, adversarial attacks it	Reading regression as a scope rather than as the reason a test exists

When the scenario says	The answer is usually	The trap is usually
What must exist before drafting a guide section, two wanted	The audience with the questions it answers, and the section outline	Translation, version control or validation, every one of which consumes drafted text

The habit worth carrying: ask what this **evaluation would have to show** for you to **stop the release**. If nothing would, it is a report rather than a gate.

What to Remember

- **Define success criteria first**, then design the evaluation that measures against them. Good criteria are **specific, measurable, achievable and relevant**, with achievability anchored to **industry benchmarks, prior experiments, research or expert knowledge** rather than to a target **beyond current frontier capability**.

- A well-formed criterion pins **three things: the threshold, the sample size, and the instrument**. The documented one sets a ceiling of under **0.1% of outputs**, across **10,000 trials**, judged by the **content filter**.

- A measurement is only evidence **relative to a threshold agreed before the result was known**. When somebody asks whether a system is good enough, the missing thing is a **pre-agreed standard at the task level**, not a better number.

- The grading ladder runs **code-based, then model-based, then human**. Code-based is **fastest, most reliable and extremely scalable** but **lacks nuance**; model-based is **fast, flexible and scalable** and must be **tested for reliability before scaling**; human is the **highest quality** and to be **avoided if possible** because it is slow and expensive. **Climb only when the rung below** cannot make the judgement.

- A criterion **requiring judgement to assess** does **not become unmeasurable**. It **moves up the ladder**.

- Only **two things must exist** before a run is possible: the **measurement definition** and the **labelled dataset**. Scoring, expert review and dashboards all **consume the run's output**, so choosing one as a prerequisite is **choosing a consumer**.

- A score generalises only when the **evaluation population matches the deployment population**, so a coverage gap is **closed before release**. Confidence intervals, caveats and segmented reporting **change how the score reads** and **leave the gap intact**.

- A trustworthy evaluation set is built from items **not derived from the artifacts under test**: **observed real demand** plus **deliberately hostile coverage**. Items spun out of the system's existing inputs are **circular**, and items written from the authors' expectations **measure the imagination that produced the gap**.

- A pre-launch framework covers **two axes that do not overlap**: **whether the thing does the job it was built** for, and **whether it holds together when attacked**. **Neither substitutes** for the other. Sampling known-good outcomes **measures nothing**, and anything that runs post-launch is **monitoring rather than a gate**.

- An automated judge stays trustworthy by **scoring every output rather than a sample**, and by being **re-examined on a schedule**. **Either habit alone** lets standards **drift unnoticed**.

- Batch runs evaluations at **50% of standard pricing** with **increased throughput**, mixing request types freely. **Order is not guaranteed**: match results using the **caller-supplied custom identifier**. **Four result types** exist, **succeeded, errored, canceled and expired**, where **expired means the request never reached the model**, and results **stay available for 29 days**.

- Structured output compliance breaks in **two documented cases**: a **refusal**, and **hitting max_tokens**, which **cuts the structure off before completion** while remaining **partially parseable**.

- Refusals arrive **three ways**: a **refusal stop reason** from streaming classifiers, a **400** when input or copyright validation fails, and an **ordinary text response** when the model itself declines. **Handle a refusal as its own outcome**, because **monitoring built on error rates will never show** it.

- **Five test types, each anchored to something different**: **unit** to one component with its dependencies **stubbed**, **integration** to the **contract between two components**, **smoke** to the **critical path run fast** to decide whether the build is worth testing further, **regression** to **behaviour that already worked**, **adversarial** to the system under **deliberate attack**. The type names **why a test exists**, not **how much it touches**.

- Before **drafting a section of an architecture guide**, two things must already exist: the **audience with the questions that section answers**, and the **section outline**. Translation, version control and validation all **consume drafted text**, so each is a **consumer wearing a prerequisite's costume**.